

WEEK 03

Unity 기초와 게임 루프

씬 · 게임오브젝트 · 컴포넌트 구조, C# 스크립팅 입문

컴퓨터그래픽스 VF3500076 · 디자인엔테크놀로지전공

조성호 · 2026학년도 2학기

도입 질문

똑같은 스크립트, 한 글자도 같은 코드

내 노트북에서는 천천히 걷고
옆자리 데스크톱에서는 뛰어감

이유는?

도입

학습 목표

1 구조

Unity에서의 제작 = 부품(컴포넌트)
조합

2 함정

AI가 빠뜨리는 단어 하나 → 게임 속도
가 PC 성능에 종속

3 VR

프레임 드랍 : 일반 게임은 불편, VR은
구역질

PART I · 60분

Unity의 기본 구조

게임오브젝트, 컴포넌트, 초당 60번 반복되는 게임 루프

게임오브젝트 = 빈 상자

Unity의 `GameObject` : 자체 기능 없음

이름과 위치만 가진 빈 상자

화면에 표시 → `MeshRenderer` 추가

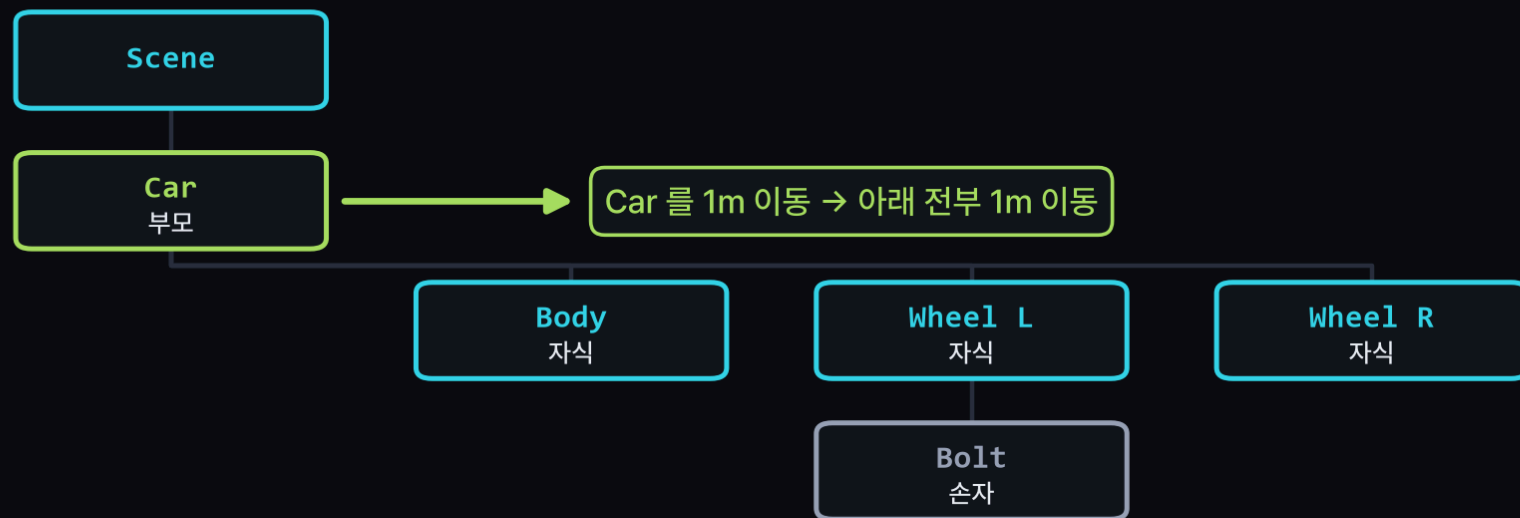
낙하 → `Rigidbody` 추가

Unity에서의 제작 = 필요한 부품을 골라 붙이는
과정

디자인 툴에서 레이어에 효과를 얹는 방식과 유사

씬 그래프 — 2주차 행렬 곱의 연쇄

씬 그래프 — 계층은 곧 변환의 연쇄

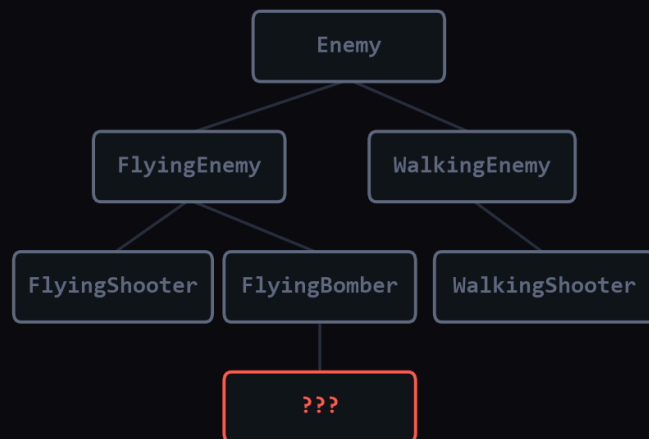


월드 위치 = $M(\text{Car}) \cdot M(\text{Wheel L}) \cdot M(\text{Bolt}) \cdot (0,0,0)$

계층 = 행렬 곱의 연쇄 — 2주차에 배운 곱셈의 적용

상속 대신 조합

상속 — 조합 증가 시 붕괴



날면서 폭탄도 쏘는 적은?

클래스 수 기하급수적 증가

컴포넌트 — 조립



필요한 부품만 붙이고 떼기

조합이 늘어도 부품 수는 그대로

Unity 에서의 제작 = 부품(컴포넌트)을 붙이는 작업

내 코드도 부품

```
using UnityEngine;

public class Spinner : MonoBehaviour // ← 이 상속이 "부품 자격증"이다
{
    public float speed = 45f;

    void Start() { } // 씬에 등장할 때 한 번
    void Update() { } // 매 프레임
}
```

MonoBehaviour 상속 → 인스펙터에 붙일 수 있는 부품

public 변수 → 인스펙터에 입력칸으로 노출

게임 = 무한 루프

```
while (게임 실행 중) {  
    입력 읽기  
    세상 상태 갱신  
    한 장 그리기  
}
```

루프가 초당 60번 반복 · 한 바퀴 = 한 프레임

`Update()` = 루프 한 바퀴에 끼워 넣는 내 코드

무거운 코드 → 게임 전체 속도 저하

한 프레임의 실행 순서

한 프레임의 실행 순서

각 단계에 넣을 코드의 선택 = 초보와 숙련자의 차이



함수별 역할

함수	호출 시점	용도
Awake	객체 생성 직후 1회	자기 자신 초기화, 컴포넌트 캐싱
Start	첫 Update 직전 1회	다른 객체를 참조하는 초기화
FixedUpdate	0.02초마다 고정	Rigidbody 를 미는 모든 코드
Update	매 프레임	입력 처리, 게임 규칙, 타이머
LateUpdate	모든 Update 후	카메라 추적 , 다른 대상을 따라가는 코드

카메라 추적을 **Update** 에 넣으면 **화면이 미세하게 떨림**

원인 : 추적 대상이 아직 이동 전일 가능성 → 실행 순서를 알아야 해결 가능

일정하지 않은 프레임 시간

한 프레임에 걸리는 시간은 매번 다름

- PC 성능
- 화면에 그릴 대상의 양
- 다른 프로그램의 작업량

"매 프레임 1cm 이동" 방식 → 속도가 프레임률
에 종속

좋은 PC = 빠른 캐릭터 → 도입 질문의 답

Time.deltaTime — 지난 프레임이 걸린 시간



AI 생성 코드에서 가장 흔히 누락되는 한 단어 — "내 컴퓨터에서는 되는데" 문제의 주요 원인

시간 · 3 ★

프레임롤 조절

인터랙티브 데모

이 자리에는 직접 조작하는 3D 데모

HTML 슬라이드를 브라우저로 열어 확인

AI가 빠뜨리는 단어 하나

AI가 자주 작성하는 코드

```
void Update() {  
    transform.Translate(  
        Vector3.forward * speed);  
}
```

// 프레임마다 speed 만큼
// = PC 성능이 곧 속도

필요한 코드

```
void Update() {  
    transform.Translate(  
        Vector3.forward * speed  
        * Time.deltaTime);  
}
```

// 초당 speed 만큼

이 함정은 화면에서 잘 드러나지 않음 — 개발자 PC에서는 정상 동작
프레임률을 바꿔 측정해야 확인 가능 → 오늘 배울 검증법

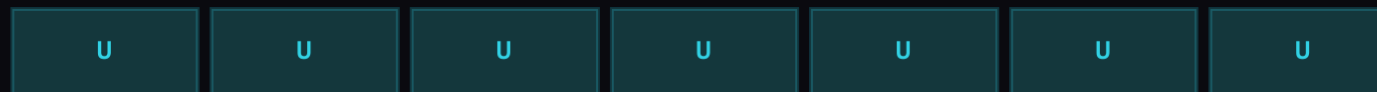
물리 계산의 별도 시계

Update 와 FixedUpdate 의 서로 다른 시계

U = Update (프레임마다)

F = FixedUpdate (0.02초마다 고정)

60 fps



30 fps



FixedUpdate



프레임률이 바뀌어도 FixedUpdate 간격은 고정

→ 어느 PC에서든 물리 계산 결과 동일

세 가지 시간 속성

속성	의미	용도
<code>Time.deltaTime</code>	지난 프레임 이 걸린 시간	<code>Update</code> 안의 모든 이동·회전
<code>Time.fixedDeltaTime</code>	항상 0.02초 (고정)	<code>FixedUpdate</code> 안
<code>Time.time</code>	게임 시작 후 흐른 총 시간	주기적 동작, 애니메이션 위상

`FixedUpdate` 안에서 `Time.deltaTime` 사용 시 Unity가 자동으로 `fixedDeltaTime` 값 반환
 헛갈리면 `deltaTime` 사용 — 두 경우 모두 정상

Update 안에서 피할 코드

```
// 나쁨 - 매 프레임 컴포넌트를 새로 찾는다
void Update() {
    GetComponent<Rigidbody>().AddForce(Vector3.up);
    GameObject.Find("Player").transform.position = p;    // 더 나쁨
}

// 좋음 - 한 번 찾아서 들고 있다
Rigidbody rb;
void Awake() { rb = GetComponent<Rigidbody>(); }
void FixedUpdate() { rb.AddForce(Vector3.up); }
```

Find : 씬 전체 탐색 → 오브젝트 1,000개면 매 프레임 1,000번 비교
60fps 기준 초당 6만 번

PART II · 30분

VR 에서의 시간

프레임 드랍 : 일반 게임은 불편, VR은 멀미 증상

프레임 끊김

일반 게임 → 짜증

VR → 구토

차이의 원인은?

화면에 표시되는 과거의 장면

고개를 돌린 그 순간의 장면은 아직 렌더링 전

센서 읽기 → 계산 → 렌더링 → 화면 점등까지 시간 소요

모니터 게임 : 지연을 **손**이 감지 — 입력 반응 지연

VR : 지연을 **전정계**가 감지 — 세상이 미끄러짐

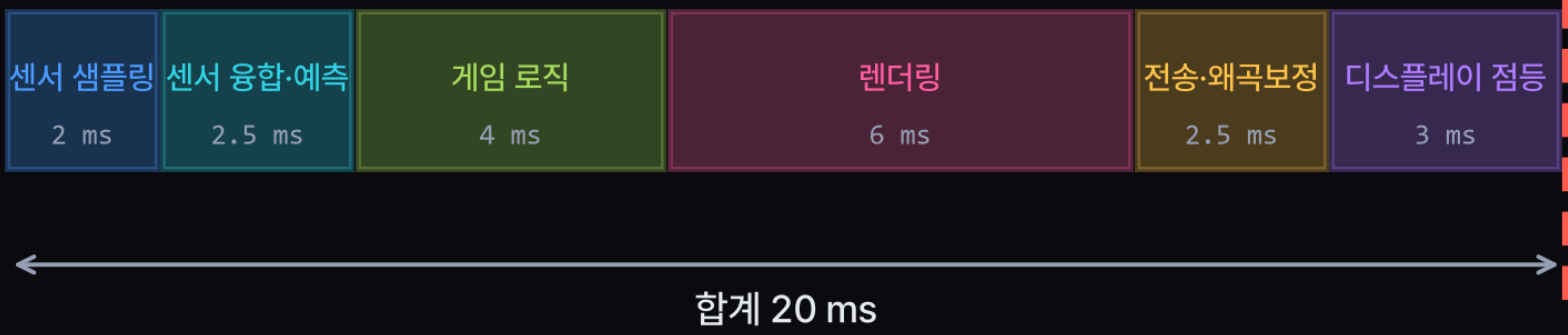
2주차 감각 충돌과 같은 원리 · 이번 원인은 **시간**

motion-to-photon

motion-to-photon latency

고개 회전부터 해당 장면의 화면 표시까지

20 ms — 이 선을 넘으면
대부분의 사람이 멀미



높은 프레임률과 짧은 지연은 별개 — 90fps 여도 지연이 길면 멀미 발생

자연 시각화

인터랙티브 데모

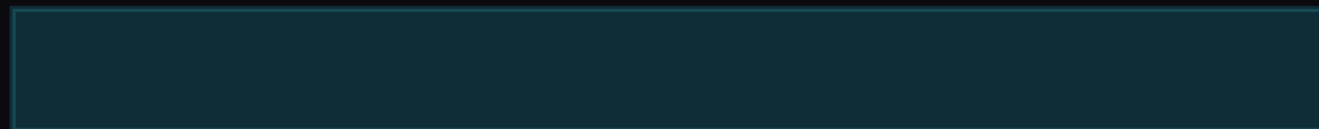
이 자리에는 직접 조작하는 3D 데모

HTML 슬라이드를 브라우저로 열어 확인

VR의 엄격한 프레임 예산

VR 의 엄격한 프레임 예산

일반 게임 60 Hz



16.7 ms

VR 90 Hz



왼쪽 눈

오른쪽 눈

11.1 ms

한 프레임에 같은 장면을 두 번 렌더링 → 실질 예산은 눈당 약 5.5 ms

예산 초과 → 프레임 전체 지연, 헤드셋이 직전 프레임을 비틀어 대신 표시(재투영)

그 순간 세상이 흔들림 — 12주차 최적화의 출발점

프레임을 놓쳤을 때

일반 게임

프레임 지연 → 늦게 표시
화면이 잠깐 끊김 — 불편한 수준

VR

헤드셋은 대기하지 않음
직전 프레임을 **비틀어** 대신 표시 (재투영)
그 순간 세상이 미끄러짐

재투영 = 구조를 지키는 안전장치, 성능 보완 수단으로 부적합

자주 발생 시 사용자는 원인을 모른 채 "이 게임 하면 어지럽다" 는 반응

설계 원칙

VR의 **Update()** :
11.1ms 예산 안에서 실행

11.1ms 안에 게임 로직 · 물리 · 두 번의 렌더링 전부 포함

Find 하나, 정렬 한 번 → 프레임 지연 유발

«일단 동작하게 만들고 나중에 최적화» 방식이 VR에서 통하기 어려운 이유

PART III · 50분

프롬프트 실습

이번 주 핵심 : 검증 조건 작성법

명세의 네 칸

[목표]

무엇을 만드는가

[제약]

무엇을 하지 말아야 하는가

[검증 조건]

어떻게 확인할 것인가

[비목표]

무엇은 만들지 않는가

이번 주 집중 : 세 번째 칸

지난주 함정(짐벌락) : 화면에서 확인 가능

이번 주 함정(deltaTime) : 화면에서 확인 불가

보이지 않는 결함 → 측정 방법을 미리 정하지 않으면 발견 불가

보이지 않는 결함의 측정

```
// 프레임률을 강제로 낮춰서 같은 코드를 두 번 돌린다
void Awake() {
    QualitySettings.vSyncCount = 0;           // vsync 를 꺼야 먹힌다
    Application.targetFrameRate = 30;        // 그 다음 30 / 120 으로 바꿔가며
}

// 10초 뒤 위치를 찍어서 비교한다
void Start() { Invoke(nameof(Report), 10f); }
void Report() { Debug.Log($"{Application.targetFrameRate}fps → {transform.position}"); }
```

30fps 와 120fps 에서 10초 뒤 위치 비교

같으면 통과, 다르면 `deltaTime` 누락 → 이것이 검증 조건

오늘의 실습 — 50분

#	실습	확인할 개념
1	AI에게 검증 조건 없이 회전·이동 스크립트 요청 · 첫 응답의 <code>deltaTime</code> 포함 여부 확인	명세의 빈틈
2	위 측정 코드로 30fps / 120fps 비교 , 결함 확인	보이지 않는 결함의 측정
3	검증 조건을 넣어 다시 요청, 응답 차이 비교	명세에 따른 결과 변화
4	공을 <code>Update</code> 와 <code>FixedUpdate</code> 에서 각각 밀어 차이 비교	고정 타임스텝
5	카메라 추적을 <code>Update</code> 와 <code>LateUpdate</code> 양쪽에 넣어 떨림 재현 (확장)	실행 순서

1~4 필수, 5 확장 · 제출물 없음, 수업 안에서 완료

이번 주 명세 예시

[목표]

큐브를 월드 **Z**축 방향으로 초당 3m 등속 이동시킨다.

[제약]

- 프레임률과 무관하게 같은 속도일 것
- **Rigidbody** 는 쓰지 않는다 (**Transform** 직접 이동)

[검증 조건]

- `vSyncCount=0` 으로 두고 `targetFrameRate` 를 30 과 120 으로 바꿔
각각 10초 뒤 `position.z` 를 로그로 찍었을 때
두 값의 차이가 0.1m 이내일 것

[비목표]

- 입력 처리, 충돌, **UI** 는 만들지 말 것

[검증 조건]에 숫자와 방법 포함

«잘 동작할 것» 은 검증 조건으로 부적합

한 장 요약

개념	핵심
게임오브젝트	빈 상자 · 부품을 붙여야 기능 발생
컴포넌트	상속 대신 조합. 조합이 늘어도 부품 수는 그대로
씬 그래프	계층 = 행렬 곱의 연쇄 · 부모 이동 시 자식 전체 이동
게임 루프	Update = 초당 60번 반복되는 루프에 끼워 넣는 내 코드
실행 순서	물리는 FixedUpdate, 추적은 LateUpdate
deltaTime	누락 시 PC 성능이 곧 게임 속도. AI가 가장 자주 빠뜨림
검증 조건	보이지 않는 결함 → 측정 방법을 미리 정해야 발견
motion-to-photon	높은 프레임률과 짧은 지연은 별개. 20ms가 경계
VR 예산	90Hz = 11.1ms. 그 안에 두 눈 모두 렌더링

다음 주

렌더링 파이프라인

정점 하나가 화면의 픽셀이 되는 과정

HMD 렌즈를 위해 그림을 미리 일그러뜨리는 이유